G-Learned Index: Enabling Efficient Learned Index on GPU

Anonymous Author(s)

ABSTRACT

AI and GPU technologies have been widely applied to solve big data problems for web systems. The total data volume worldwide reaches 200 zettabytes in 2022. How to efficiently index the required content among massive data becomes serious. Recently, a promising learned index has been proposed to address this challenge: It has extremely high efficiency while retaining marginal space overhead. However, we notice that previous learned indexes have mainly focused on CPU architecture, while ignoring the advantages of GPU. Because traditional indexes like B-Tree, LSM, and bitmap have greatly benefited from GPU acceleration, a combination of a learned index and GPU has great potentials to reach tremendous speedups.

In this paper, we propose a GPU-based learned index, called G-Learned Index, to significantly improve the performance of learned index structures. The primary challenges in developing G-Learned Index lie in the use of thousands of GPU cores including minimization of synchronization and branch divergence, data structure design for parallel operations, and usage of memory bandwidth including limited memory transactions and multi-memory hierarchy. To overcome these challenges, a series of novel technologies are developed, including efficient thread organization, succinct data structures, and heterogeneous memory hierarchy utilization. To the best of our knowledge, this is the first GPU-based learned index. Compared to the state-of-the-art learned index, the proposed G-Learned Index achieves an average of 3050× speedup. Meanwhile, we attain 34× less query time over the state-of-the-art GPU B-Tree. We have made G-Learned Index available at https://anonymous.4open.science/r/G-Learned-Index-8D89.

CCS CONCEPTS

• **Information systems** → **World Wide Web**; • **Computing methodologies** → *Machine learning*; **Parallel computing methodologies**.

ACM Reference Format:
Anonymous Author(s). 2023. G-Learned Index: Enabling Efficient Learned Index on GPU. In *WWW '23: Proceedings of the ACM Web Conference 2022, April 30 - May 4, 2023, Austin, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/1122445.1122456

1 INTRODUCTION

The currently increasing amounts of data for Web systems pose significant hurdles in terms of efficient data accesses [14, 28, 34, 35, 49]. For example, the total data volume stored worldwide reaches 200 zettabytes in 2022 [4], and 11.45 zettabytes of data are generated everyday [8]. Index structures such as B-Tree [19], hash tables [32], and Bloom filters [5] are required for efficient data access. In recent years, Kraska et al. [25] proposed a promising transformative understanding of the index problem – Indexes are models that can be trained to map keys to their locations in a sorted order. More specifically, the position of a key can be estimated in a sorted array if the cumulative data distribution function f of the given dataset can be learned, and accordingly, traditional index structures can be replaced by f. This "learned" approach achieves significant time and space benefits compared to traditional indexes. Further, GPUs are effective and popular accelerators for index structures [7, 21, 22] and machine learning [13, 24, 26]. In particular, the contexts of batch operations in dynamic scenario are perfectly suited for GPU parallelism. In fact, many index structures, such as B-Tree [7], have already been proven to be successfully accelerated by executing batch operations on GPUs, bringing even more than 60× performance improvement [7]. Therefore, we consider it essential to enable the learned index on GPUs to further improve its performance [6, 41, 48].

Enabling efficient learned indexes on GPUs has three major advantages. First, thousands of GPU cores offer huge potentials for dramatically increasing the performance of learned indexes. For example, NVIDIA GeForce RTX 2080 Ti GPU [11] contains 4352 cores, which is hundreds of times the number of common CPU cores [10]. Second, comparable workloads, such as B-Tree, matrix computation, and machine learning, have already benefited significantly from GPU acceleration. For example, research [6] shows that GPU LSM can obtain a 13.5× performance boost from GPU. Higher gains in learned index performance can be expected because they advance intelligent structure design in terms of time efficiency and more crucially, space savings. Third, GPUs have been widely used in database operations, and the GPU-based learned index can operate as a sub-component that expands the possibilities for GPU query engines, e.g., MapD [44]. Eventually, we can use an epoch based strategy to integrate our index into a full-fledged database system [33, 42, 50], an approach that has been widely used in database systems, e.g., BatchDB [36]. The basic idea is to split the queries in epochs and concurrently process the queries in each epoch.

We show a use case of applying GPU-based learned index in the area of web search and information retrieval in Figure 1 [14, 28, 34, 49]. Requests for information are generated on the user end and are gathered and put into the central search engine. The search engine then issues a query for each request trying to retrieve information in the web pool. This data pool is structured through an inverted index that can benefit from our GPU-based learned index. The targeted web pages are then indexed and retrieved based on the user's information need. Other application scenarios, such as online ride hailing services [20], require streaming query requests to the central server and indexing the database for available calls

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW '23, April 30 - May 4, 2023, Austin, USA
© 2023 Association for Computing Machinery.
ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/10.1145/1122445.1122456

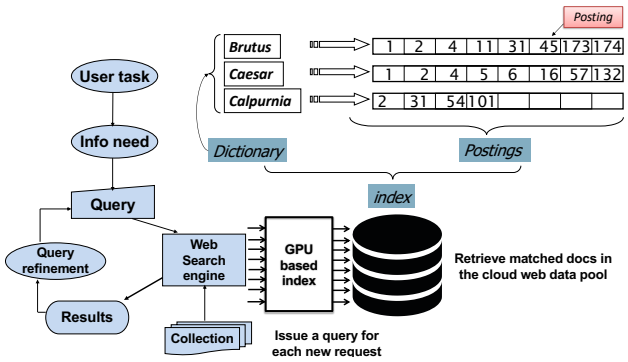

Figure 1: Use case of GPU-based index in web search and information retrieval.

in order to make responds. These tasks all involve high volumes of concurrent queries and thus can benefit from GPU index with high throughput.

Although enabling learned index on GPUs is both advantageous and necessary, developing an efficient GPU-based learned index faces three major challenges.

First, the performance of GPU-based learned index is highly dependent on the GPU's thousands of cores being fully utilized. This poses substantial difficulty on GPU-based learned indexes because learned indexes entail fundamental dependency between operation phases that is not ideal for parallelism. Namely, for a query, operations on the next level in the recursive structures are dependent on the output of the model in the upper level of learned index [16], which is extremely unfriendly to the SIMT GPU working pattern [31].

Second, the multiple levels of learned index need to adapt to the complicated GPU memory hierarchy. The size of learned indexes varies depending on the dataset and the precision loss ϵ [16]. For example, a huge dataset with a large ϵ can also result in an index structure that is compact enough to fit into the shared memory on GPU. In contrast, if a small ϵ is applied, the produced structure of even a relatively small dataset has to be located on global memory. Therefore, the configurations of learned indexes need to be adjusted.

Third, the complicated index structures and operation algorithms pose difficulties to space occupation and time complexity. In terms of space concerns, a complete index structure can be too large to put onto a GPU device due to space constraints. For example, GPU cannot hold index structures with more than millions of elements. As for execution time, query operations rely on recursive searches for learned indexes, which is time-consuming.

There is a large literature of learned indexes [29], but none can address the above challenges in applying learned index on GPUs. In detail, there are currently two mainstream learned indexes that successfully achieve significant time and space savings: the Recursive Model Index (RMI) [25], an innovative hierarchical structure integrated with machine learning models, and the Piecewise Geometric Model index (PGM-index) [16], which uses algorithmically generated linear regression models to approximate key positions. More recent studies have stated that models on the bottom level of the learned index can be replaced by B-Trees to maintain a balanced partition and improve precision [12, 30, 41]. However, these studies of learned index are mostly concerned with the designs on CPU architectures.

In this paper, we propose a **G**PU-based **learned index**, called G-Learned Index, which enables efficient learned indexes on GPUs. G-Learned Index incorporates three innovative features and can address the three challenges listed above. First, we create a fine-grained synchronization-free approach for processing batch queries with low branch divergence. Second, we develop a novel heterogeneous model to maintain a succinct structure on GPUs and a lock-step working pattern for various query operations. Third, to achieve maximum bandwidth efficiency, we adaptively separate the structures of G-Learned Index and place them into distinct hierarchical memories on the GPU.

We evaluate G-Learned Index on four real-world datasets and four synthetic datasets with different data types and key sizes. Experiments show that G-Learned Index achieves an average of 3050× speedup in throughput over the state-of-the-art learned index. Compared to the state-of-the-art B-Tree on GPU, G-Learned Index achieves 34× performance speedup on average, and saves orders of magnitudes of space on various workloads.

To the best of our knowledge, this is the first work that enables efficient learned indexes on GPUs. Specifically, we make the following contributions in this paper.

- We develop the first dynamic learned index on GPUs with an intelligent design that is optimized for queries and updates.
- We design an efficient indexing strategy that achieves coalesced memory access and reduced branch divergence.
- We extensively assess G-Learned Index on eight datasets and demonstrate its significant advantages over the state-of-the-art methods.

2 BACKGROUND

2.1 Learned Index Structures

Learned indexes combine the index structures with the idea of machine learning, and they regard indexes as trainable models. More precisely, assume that we query the position of the key k in the sorted array A; indexes are viewed as models with k as the input and predict the position of k in A. To accomplish this, the Cumulative Distribution Function (CDF) f of A must be learned, and the position of k can be obtained by multiplying N by $f(k)$, where N denotes the size of A. To assist the learning process of the function, models ranging from the linear functions to more sophisticated machine learning strategies have been explored [16, 18, 25]. Currently, the best-known learned index models are the Recursive Model Index (RMI) [25] and the Piecewise Geometric Model (PGM-index) [16].

Recursive Model Index (RMI). RMI is a multi-level recursive regression model that is organized as a hierarchical Directed Acyclic Graph (DAG). Each node in the graph represents a model that behaves like a CDF. When a key k is queried, we obtain the model denoted by the node in the first level. Note that there is only one node in the first level. We take k as input and keep the model's output, which is used to select the model for the next level. By repeating these operations, we can obtain the output of the entire RMI at the last model in the last level. The output can be interpreted as the estimated position of k in the array A. Finally, we perform the "last mile" search to determine the real location of k.

Piecewise Geometric Model index (PGM-index). The PGM-index is another learned index structure that occupies a prominent place with its succinct structure for constructing algorithmic models to learn data distributions [16]. By solving the simplified Piecewise Linear Approximation (PLA) problem, the PGM-index develops a Fitting-Tree [18], which provides a more specific and interpretable method to learn the data distribution. In other words, for a given deviation bound $\epsilon \geq 1$, the Fitting-Tree generates a combination of linear models [18]. Specifically, the optimal PLA finds a minimum number of linear model groups for a given array $A[0, b]$, such that each linear model is responsible for an indexing subset $A[a_i, b_i]$. Each linear model is referred to as a *segment*. Note that a *segment* is a tuple of three elements, $(key, slope, intercept)$. For an input $A[x], x \in [a_i, b_i]$, the corresponding linear model returns its estimated position y, which guarantees $|y - x| \leq \epsilon$. Satisfying the optimization principle and Markov property, an $O(n^3)$ Dynamic Programming (DP) algorithm is generated to solve this PLA problem, whereas it admits a streaming algorithm in linear optimal space and time after extensive study [38].

For a query specified by x, the PGM-index retrieves the visited segment in the current level and uses it to estimate the position of x of the next segments. A binary search inside the 2ϵ range centered on the estimated position is then used to determine the accurate position.

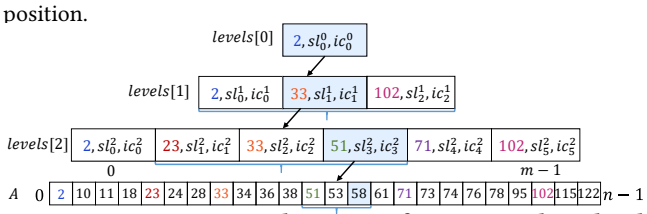

Figure 2: A recursive search process of x = 58 in a three-level PGM-index with ϵ = 1 .

Example. An example of PGM-index query $x = 58$ with $\epsilon = 1$ is shown in Figure 2. At the highest level, we have $\lfloor x \cdot sl_0^0 + ic_0^0 \rfloor = 1$. Then, in $levels[1][1 - \epsilon, 1 + \epsilon]$, we look for x inside the key range $[2, 33, 102]$. Following a binary search, the segment $levels[1][1]$ (encircled by the blue bracket) is returned because x falls within 33 and 102. We conduct a similar process by first calculating $\lfloor x \cdot sl_1^1 + ic_1^1 \rfloor = 2$ and finding segment $levels[2][3]$ in $levels[2][2 - \epsilon, 2 + \epsilon]$ among keys $[23, 33, 51]$. Up to now, we take four procedures to reach the segment at the lowest bottom level. In the lowest level, we have $\lfloor x \cdot sl_3^2 + ic_3^2 \rfloor = 12$. A search process is then performed in $A[12 - \epsilon, 12 + \epsilon]$ among the keys $[51, 53, 58]$ before the accurate position of 13 is identified.

2.2 General-Purpose Computing on GPU

A Graphics Processing Unit (GPU) is a specialized electronic device used for processing graphics. Currently, GPUs can also process applications that were previously handled by CPUs. A GPU contains several Streaming Multiprocessors (SMs), and each SM contains a large number of light-weight cores. Threads are grouped into blocks, which are then assigned to SMs. During execution, the threads in each SM are executed in *warp* granularity, which means that threads within the same warp must be executed in a single-instruction-multiple-threads mechanism (SIMT). Furthermore, the GPU memory hierarchy is complex. Individual threads, for example, have their local registers. Threads in the same block can access the fast shared memory, which has a limited amount of space. There are also more peripheral storage areas, including L1 and L2 caches located independently on each SM, and a global memory that is accessible to all threads but has a slower transfer speed.

GPU applications. GPUs have been successfully used in a variety of applications, including data science and machine learning. For example, GPUs have been used in data management systems [17]. Besides, traditional index data structures, such as B-tree [7], B+-tree [22], and LSM tree [6], all can be accelerated by GPUs. Machine learning workloads such as linear regression models [27] and neural network models, including fully connected networks [47], recurrent neural networks [43], and convolutional neural networks [45], can all benefit from GPU acceleration. Hence, we believe that learned indexes also have significant acceleration potentials, although challenges must be overcome.

3 MOTIVATION

3.1 Developing Learned Index on GPUs

Guidelines: First, granularity should be an important factor. Second, diving into operator-level and calculation-level design strengthens the efficacy of the GPU-based models. Third, the feasibility and flexibility of thread assignment working on the structure should be considered.

Idea. As discussed in Section 1, it is necessary to develop learned index on GPUs. To fully leverage GPU resources, our basic idea is to process queries in parallel. Building machine learning models is well suited for GPUs because matrix multiplications in both training and inference phases can significantly benefit from GPU's high parallelism. Accordingly, our first attempt is to launch and run multiple GPU threads and make them work cooperatively for a single query. However, coordinating different threads under a single query is much harder than a one-thread-one-query working pattern. Furthermore, the overhead of transferring data between CPU and GPU could not be neglected.

The second approach is to assign each query to a single thread and index the queries concurrently. In other words, we process indexes in queries at batch granularity. Due to massive numbers of GPU threads, GPU index has greater potentials to achieve higher query throughput than CPU-based learned indexes. Accordingly, we need to construct a structure that matches the indexing strategy. Also we need to support efficient data transmission between CPUs and GPUs, because data transfer is an important factor affecting performance.

Comparison of RMI and PGM-index. We next analyze the index options, specifically, RMI and PGM-index. According to the SIMD working pattern, the key for efficient performance is to execute the same operations for different GPU threads. In the RMI [25] inference phase, different queries follow varying searching patterns, arriving at different nodes with diverse machine learning models. Operations conducted on respective threads differ from each other, which causes branch divergence, making it hard to fully leverage the GPU resource. Further, the large number of fine-tuned parameters in various RMI machine learning models leads to more uncertain models. In contrast, the PGM-index [16] has only three parameters for a linear model, and maintains a succinct index structure. These attributes make it more suitable for GPUs.

Identifying PGM-index as our focus. In conclusion, we develop a GPU-accelerated learned index based on the PGM-index mainly for two reasons. First, the PGM-index structure is relatively simple and enables efficient data movement between CPUs and GPUs. In detail, it is constant in space for describing each segment and constant in time for progressing to the next level of segments. In addition, the PGM-index has a low search cost that scales logarithmically with ϵ. Second, the PGM-index lends itself to parallelism. In particular, each step involves the same operations between different threads. This is an ideal characteristic for GPU parallel computing to bring out the optimal of GPU efficacy. Moreover, the PGM-index can guarantee a precision loss of ϵ.

However, choosing PGM-index as our focus does not preclude the possibility of building GPU-based learned index for the kind of RMI that uses *linear segments* in all its models. This is because *linear segments* also mean constant space and reduce branch divergence. Since this does not differ much from PGM index, the designs we formulate on GPUs for PGM index can be easily adapted to the linear segment based RMI.

4 OVERVIEW OF G-LEARNED INDEX

We propose a GPU-based learned index, called G-Learned Index, which can enable efficient learned index on GPUs. Figure 3 depicts an overview of G-Learned Index, which consists of three major components: 1) architecture construction, 2) hierarchical memory arrangement, and 3) query execution. These components combine to generate G-Learned Index.

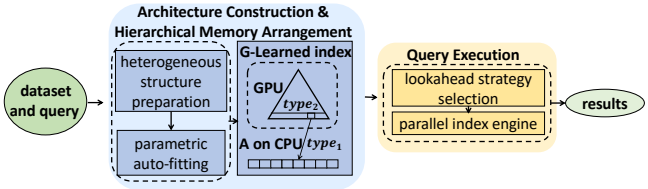

Figure 3: Overview of G-Learned Index.

The first component is **architecture construction**. The architecture comprises two phases. The first phase is *heterogeneous structure preparation*. Specifically, we place all the segments on the GPU and the original data, stored in array A, on the CPU. The initiative here is that A can be too large to fit into the GPU memory. For smaller size of A, we take the same layout for better performance, which is explored in Section 5.1. The second phase is *parametric auto-fitting*, and it is an adaptive tune-up method for refining the parameters, namely the precision loss ϵ, in the index.

The second component is **hierarchical memory arrangement**, which fits our heterogeneous model to the memory hierarchy of the GPU. In detail, we separate the storage into a hierarchy based on response time. Storing segments on GPU's shared memory results in faster data transfer because of higher bandwidth. If the size of the segments is too large, those segments exceeding the shared memory limits should be separately stored in GPU's global memory.

The third component of G-Learned Index is concurrent batch **query execution**. This is the component responsible for indexing our G-Learned Index. The query execution for concurrent batch processing should have features of advantageous mutability, accessibility, and flexibility. Specifically, we employ a *lookahead strategy selection* to determine the indexing strategy. We discuss our observations and insights in Section 5.2.

Novelties. There are three major novelties in G-Learned Index. First, it parallelizes the PGM learned index using thousands of GPU cores, considerably improving the speed of learned indexes to an unprecedentedly high throughput for batch queries. Second, our carefully constructed heterogeneous structure caters to varied features of CPUs and GPUs, taking into account both learned index structure characteristics and device architecture properties. Third, we develop a set of optimizations such as parallel construction and lookahead strategy.

Relevance to World Wide Web. Web servers can benefit from GPU-based learned index through many applications. First, *general information retrieval* [37], such as web search elaborated in Figure 1 of Section 1, can be greatly accelerated. Second, for *web high concurrency workloads* [15], e.g., Black Friday [2], GPU-based learned index can be used to handle such batching queries. Third, for *machine learning feature stores* [3], analysts need to retrieve training and testing records from diverse datasets among different tasks through web servers [39]. We can use GPU-based learned index to select training and testing records concurrently.

We next show the detailed design of G-Learned Index.

5 DETAILED DESIGN

In this section, we first describe how the proposed heterogeneous architecture is built and indexed using succinct structures in Sections 5.1 and 5.2. Then, we specify the data structures we employ in Section 5.3, accompanied by the discussion of insertions and deletions in Section 5.4.

5.1 Architecture Construction

Heterogeneous structure preparation. In this module, we split the index into two parts and distribute them to distinct devices. We start by conducting the part-division job. The first part is levels of segments generated by the PLA-model [16, 18], and the second part is a dynamically allocated container array A. This split design takes advantage of the following insight. Although a PGM-index is able to index enormous datasets, G-Learned Index is constrained by the limited capacity available on GPUs. Fortunately, we observe that even when a large amount of space is required for array A, the last level of segments never takes up much space since the number of segments in that level is empirically small (a theoretical bound is $n/(2\epsilon)$). As a result, we learn that there is no need to load the entire array A to the GPU. Therefore, we divide G-Learned Index into two parts — the array A storing keys in the original dataset, and segments produced by the PGM index — and place A on CPU and segments on GPU. For smaller size of A, we take this same layout because it can achieve better performance. The reason is that the binary search range is larger on A than on any other segment levels, and binary search among large range is not efficient on the GPU.

Parametric auto-fitting. We focus on optimizing the index in this parametric auto-fitting module, which mainly depends on the choice of ϵ to control the number of segments appearing in our index. The number of segments has a significant impact on space occupancy and query performance. Meanwhile, a varying ϵ itself can influence the search time when pinpointing the accurate position in the range of length 2ϵ. Therefore, the complexity of time and space is complicated in relation to ϵ. In our design, we

first conduct theoretical inspection and then obtain the optimal ϵ. The theoretical inspection is divided into two steps. In the first step, we look into the procedures it takes for indexing a key k in the model and how the choice of ϵ can affect those procedures. In the second step, we provide detailed analysis to support the division of the procedures theoretically. The full analysis is available in Appendix D.

Parallel PLA generation. We use the PLA model [16] to generate the segments that cover the array A. Now, we explore the possibilities for building the segments in parallel.

We apply an $O(n)$ greedy algorithm in PLA for constructing the ϵ-approximate segments for A, where ϵ is given as a parameter. First, we create a Cartesian plane containing points $(k_i, rank(k_i))$ for all k_i in array A. Second, starting from the leftmost point to the rest of points on the right in order, a convex hull is maintained for a set of points until the height of a rectangle enclosing the convex hull exceeds 2ϵ. Then, we obtain a *segment* with corresponding $slope, intercept$, and key (key denotes the value of the leftmost point, which is k_1 here) for the points in the set. This process is repeated until all *segments* are created.

We then analyze the construction overhead brought by splitting the array, namely, separately constructing PLA-model and then merge back them to one model. We come to the following theorem.

Theorem 1. *Let P be divided into $\omega+1$ sections, each running PLA-model in parallel before the results are merged together to complete the segment generation ($\omega \geq 1$). This adds at most ω segments on top of the segments needed when we perform the PLA-model on the whole P.*

The proof can be found in the Appendix A.1. Theorem 1 shows that we can benefit from parallel construction. In fact, Parallel computing has been shown in experiments to reduce the construction time of the PLA-model from a few seconds to less than a second.

5.2 Query Execution

In this part, we solve the index problem in a batch search context. We use batch granularity because we focus on bulk queries, which can benefit from thousands of GPU cores to index, resulting in massive acceleration.

General design. The parallel query operations work as follows. First, following the heterogeneous design, segment levels are built based on array A and loaded to the GPU device. Second, when bulk queries arrive, each query is assigned to a single thread that is in charge of resolving the index problem based on our heterogeneous model. Third, an approximate position is returned from GPU and the final position is pinpointed on the CPU.

Detailed design. Query operations for a single task are performed recursively in GPU from the root segment to the bottom level in G-Learned Index. At each level, a thread uses a referring segment passed down from the upper level to estimate the segment in the next level. Then, a search in a range size of $2\epsilon_2$ is performed to find the right-most segment whose key remains no larger than k. Given that the $2\epsilon_2$ search range can differ from one thread to another, this operation can lead to considerable time overhead. The reason for this is that each thread should compare its own k to a key stored in a single segment. Its time overhead cannot be hidden, especially when the number of queries running in parallel is large. Therefore, we select an ϵ_2 that is small enough ($\epsilon_2 = 4$) to compensate the overhead caused by divergent data accesses.

To further reduce branch divergence, we design a *lookahead strategy selection*. The complete details and analyses are available in Appendix B. The idea is that removing the upper levels and conducting direct traversal check search on the bottom level can achieve better query performance. This is because we need fewer memory transactions, and more importantly, perform the exact same numerical comparison between threads.

Algorithm. We show the pseudocode of the query execution of G-Learned Index in Algorithm 1, which has four major inputs: 1) pre-trained parameters of ϵ_1 and ϵ_2 in the construction phase, which can achieve high query performance, 2) $levels$ produced by PGM-index, 3) array A containing n keys, and 4) bulk query k. The workflow is as follows. First, $levels$ are transferred to GPUs (Line 24),. Second, we then conduct the *lookahead strategy selection* in Line 25, and then apply k to that choice. Third, we do queries on GPU where $type_1$ procedures are performed (Lines 2-11 and Lines 16-19). Finally, $type_2$ procedures are executed on CPUs (Lines 12 and 20).

Algorithm 1 Query execution

```
1: function completeQuery(A, n, ε1, ε2, levels, k)
2:     pos = f_r(k), where r = levels[0][0]
3:     for i = 1 to SIZE(levels) − 1 do
4:         lo = max{pos − ε2, 0}
5:         hi = min{pos + ε2, SIZE(levels[i]) − 1}
6:         s = the rightmost segment s′ in levels[i][lo, hi] such that s′.key ≤ k
7:         t = the segment right to s
8:         pos = ⌊min{f_s(k), f_t(t.key)}⌋
9:     end for
10:    lo = max{pos − ε1, 0}
11:    hi = min{pos + ε1, n − 1}
12:    return search for k in A[lo, hi]                    ▷ Processed on CPU
13: end function
14:
15: function conciseQuery(A, n, ε1, ε2, levels, k)
16:    r = SIZE(levels) − 1
17:    s = the rightmost segment s′ in levels[r] such that s′.key ≤ k
18:    lo = max{pos − ε1, 0}
19:    hi = min{pos + ε1, n − 1}
20:    return search for k in A[lo, hi]                    ▷ Processed on CPU
21: end function
22:
23: function Query(A, n, ε1, ε2, levels, k)
24:    load levels to GPUs
25:    p = arg max_{p∈P} p(k), where P = {completeQuery, conciseQuery}
26:    return p(k)                          ▷ Each thread assigned for a query task
27: end function
```

Theoretical analysis. Given an array consisting an ordered sequence of $k_i (i = 0, ..., n-1)$, the PLA-model in PGM-index is capable of determining a minimum number m_{opt} of segments for the original array (denoted as A), where $m_{opt} \leq n/(2\epsilon_1)$. For convenience, we use m instead of m_{opt}, and draw the following theorem with proof detailed in Appendix A.2.

Theorem 2. *G-Learned Index with parameters ϵ_1 and ϵ_2 indexes the array A in $\Theta(n+m)$ space and answers queries in amortize $O((\log \epsilon_1)/\omega_1 + \log \epsilon_2 (\log_c m)/\omega_2)$ time and $\Theta(\epsilon_1/B + m/(\omega_2 B))$ I/Os with $O(\epsilon_2 \log_c m)$ memory transactions on GPUs. In the above claim, $c \geq 2\epsilon_2$ is the fan-out achieved between the neighboring levels, ω_1 and ω_2 denote the granularity on CPU and GPU, respectively, and B is the block size in the external-memory model.*

5.3 Data Structures

The data structures of G-Learned Index include a hierarchical self-maintained memory, positioning data structure, and prefix trees for indexing strings.

Hierarchical memory arrangement. When performing query operations, we assign one query task to a thread, and each thread accesses the segment levels in G-Learned Index. Therefore, data retrieval efficiency influences query performance on GPU, ultimately affecting total time cost. Fortunately, we can make full use of the GPU memory hierarchy to build our index structure. Constant memory located on GPUs can be accessed by all threads on the device and has a limited space of 4 KB, whereas shared memory is a programmable cache on individual SMs and has a size of 64 KB of our platform. Our memory is managed by first loading segment levels to GPU global memory. We then let each block in an SM occupies a copy of the structure in the shared memory if the structure does not exceed the capacity of the shared memory. This takes advantage of shared memory's faster access speed compared to global memory by retrieving data directly from the shared memory for each thread. Another suboptimal choice is to use constant memory for storage pool, which can provide moderate data access speedup. The reason is that constant memory resides on the same chip as global memory, and thus fails to produce the outstanding performance of shared memory.

Data structure of positioning within the ϵ range. A search operation within an interval for each segment level on GPU whose size is at most 2ϵ ($2\epsilon_2$ specifically) is the key step for query execution on a single thread. This is achieved by a binary search in our design. The binary search outperforms a traversal search because it reduces the number of comparisons from 2ϵ at worst to a fixed $\log(2\epsilon)$. A fixed operation pattern means the avoidance of branch divergence between threads. Although with probability $p = \log(2\epsilon)/(2\epsilon)$, a traversal search behaves better than binary search (approximated to $1/2$ when ϵ is small), this value is diminished by a SIMD pattern of w threads working on a batch. Specifically, the probability is reduced to $p^w = (\log(2\epsilon)/(2\epsilon))^w$ because the processing of a batch finishes only when the last thread in that batch completes its job. For example, assume the concurrent queries be grouped in $m = 10^5$ SIMD batches. Following Chernoff bound, then, the probability that 100 out of 10^5 SIMD batches can obtain better performance on traversal search is smaller than $2e^{-m/(3*10^6 p^w)}$, i.e., $2e^{-1000}$ with $w = 16$, $\epsilon = 4$, and $m = 10^5$.

Indexing strings. Indexing string is commonly used by many databases [16, 25]. However, the existing PGM-index and many GPU-based indexes focus on only indexing numerical values [6, 7, 16]. Extending the model for strings introduces plenty of difficulties. The primary concern is how to convert strings to features of our model, a process known as tokenization. Fortunately, studies [9, 25] on modeling strings have been able to facilitate our practice. Overall, we adopt the advantage of a prefix tree formulated on the dataset dictionary. In an efficient and expressive solution, we represent a string as a feature key $u.key \in \mathbb{R}$, where $u \in T$ and T is the prefix tree generated on the given string dataset. We then follow the same efficient heterogeneous modeling as we perform for numerical values. The difference is that the input is the feature key representation of a string.

5.4 Updates in G-Learned Index

In this section, we consider random updates, including insertions and deletions, in G-Learned Index. For *insert*, it is more difficult to develop compared to traditional indexes. Specifically, we adopt the logarithmic method for inserts at arbitrary positions in the original array [16]. That is to say, we maintain $\log(n)$ containers $S_0, S_1, ..., S_k (k = \log(n))$ at any time having volumes of $2^0, 2^1, ..., 2^k$ elements, respectively. In each container, we build a G-Learned Index. When an element is inserted, we first attempt to add it to S_0. If the added element makes the total number of elements exceed the volume of the container, neighbor containers are merged in a cascade way. For example, assume S_i is the first empty container. After inserting the key x, we build a new G-Learned Index over the merged container $S_i = S_0 \cup ... \cup S_{i-1} \cup \{x\}$. Overall, we pay $O(\log n)$ amortized time per insertion. To delete a key, we add a tombstone value associated with the key x.

When indexing batch queries, we allocate $\log(n)$ threads for one query and let the threads work simultaneously. There are two levels of parallelism. To index the key x, a group of $log(n)$ threads work in parallel, with thread i indexing x in S_i, where $i = 0, ..., \log(n)$. At the same time, different thread groups work in parallel to index different keys.

5.5 Implementation

We integrate G-Learned Index and PGM-index [16] implementations together, to help users use the learned index in a variety of heterogeneous application scenarios. The G-Learned Index consists of two major parts. 1) a CPU-located part handling data input and work assignments on the CPU, and 2) a GPU-situated working kernel mainly responsible for acceleration. G-Learned Index is written in C++ and CUDA, while OpenMP has also been used to facilitate our development. With a user-friendly interface, it is easily applicable to databases and applications that can benefit web servers.

6 EVALUATION

6.1 Experimental Setup

Methodology. We compare G-Learned Index to four methods. The first method is "ALEX" [12], the most recent updatable Recursive Model Index (RMI). The second method is the PGM-index [16], which is the state-of-the-art learned index denoted as "PGM". However, we find that the original PGM is a sequential version. For fair comparison, we use OpenMP to develop a parallel PGM-index version that can take advantage of CPU's parallelism. Accordingly, the third method is a parallel version of the PGM-index, denoted as "PGM (parallel)". The fourth method is the state-of-the-art GPU-based B-Tree [7], denoted as "GpuBTree". Our GPU-based learned index is denoted as "G-Learned Index".

Datasets. We refer to SOSD [23] for evaluation datasets. SOSD is a recent proposal on benchmarking learned index, which encompasses a wide range of datasets. In detail, we use eight datasets. Four datasets come from the real world, including *Books* that contains popularity data of 200M book sales on Amazon [46], *Facebook* that represents 200M sampled IDs of users drawn from the Facebook dataset [23], *Street* that consists of 800M location information out of OpenStreetMap presented in Google S2 CellIds [40], and a string dataset *Dict* that consists of 341K English words from a

dictionary [26]. Four synthetic datasets of 0.1M, 1M, 10M, and 100M are generated randomly, denoted as "*1e5*", "*1e6*", "*1e7*", and "*1e8*", respectively. "*1e5*" is generated according to the uniform distribution in the interval $[0, u)$, "*1e6*" to the normal distribution with mean μ and standard deviation σ, "*1e7*" to the exponential distribution with parameter λ, and "*1e8*" is randomly generated. We set $u = 2^{32}, \mu = 2^{16}, \sigma = 2^{10}$, and $\lambda = 2^{-5}$. The randomly generated datasets involve a high level of uncertainty and reveal a wide range of difficult-to-identify data patterns. As a result, we believe that these datasets present challenges for learned indexes in grasping data schemes.

Platform. Our experiments are conducted on a platform equipped with an Intel i9-9900k CPU and an NVIDIA GEFORCE RTX 2080 TI GPU. The CPU has 8 cores, each can support 2 threads. The GPU has 4,352 cores of Turing architecture with a computing capability of 7.5. The GPU can achieve a maximum memory bandwidth of 616 GB/s, 0.4 tera floating-point operations per second (TFLOPS) on double-precision, and 13 TFLOPS on single-precision.

6.2 Performance

We compare the performance of various methods in terms of both throughput and latency. For each dataset, we execute 2^{22} key lookup queries. Each query is assigned with a random key. A batch size of 4096KB is used and ϵ is set to 128.

Throughput. We show the throughput of different methods in Figure 4, and have the following observations.

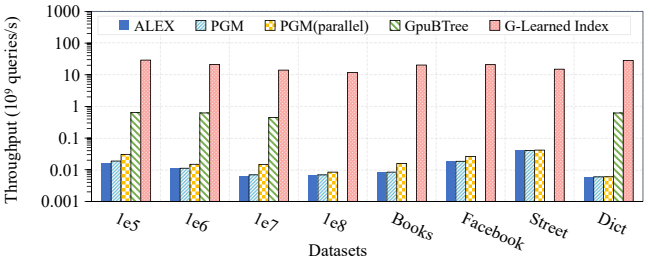

Figure 4: Throughput of different methods.

First, G-Learned Index achieves the highest throughput among all methods. It can achieve an average of 21×10^9 queries per second, which is 3050× the performance of PGM-index, and 1880× the performance of its parallel version. Compared to the state-of-the-art GPU-based B-Tree, G-Learned Index still achieves an average speedup of 34×. We run through only the four small datasets for GPU-based B-Tree, and the other datasets incur errors due to illegal memory accesses. Compared to GPU-based B-Tree, the reason for G-Learned Index's higher throughput is due to its more compact learned structure, more succinct representation, and better use of heterogeneous architecture.

Second, we discover that parallel technology is extremely effective at accelerating learned index. The CPU platform can support up to eight cores, and the parallel version of PGM-index has an average 52% higher throughput than the sequential version. GPUs provide much more lightweight cores. With proper optimizations, including coalesced access, avoidance of branch divergence, and use of memory hierarchy, as discussed in Section 5, G-Learned Index can provide a further significant performance improvement.

Third, we can see from Figure 4 that the G-Learned Index throughputs are comparable on different datasets. When G-Learned Index processes a greater number of elements, its performance stays stable (statistically from 0.4% to 9.6% reduction at most). The reason is that a *segment* can carry large volumes of information. This suggests promising scalability of G-Learned Index. However, we observe that the parallel PGM-index and GPU-based B-Tree suffer significant time efficacy loss when the number of elements to maintain is increased (from *1e5* to *1e7*). The parallel PGM-index and GPU-based B-Tree, in particular, see a reduction in throughput from *1e5* to *1e7* by 19% to 30%, and 14% to 43%, respectively. This is because the functions of the C++ STL library used in PGM-index are sensitive to dataset size. Instead, we turn them into GPU kernels, and ensure that the growing size of datasets can lead to a near-constant rise in kernel operation time.[1] This increase is then offset by the large number of batch queries. As for GPU-based B-Tree, throughput is hinged on the height of the B-Tree, prominently influenced by the expanding size of the index structure linear in the number of input keys.

Latency. We show the latency results of different methods in Figure 5. We define latency as the end-to-end time for a query to complete from the beginning to the end. For methods using parallel batching, a query ends when all queries in the same batch finish. The batch size is 2^{10}. We have the following observations.

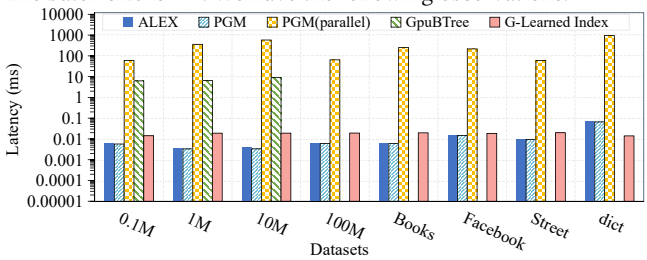

Figure 5: Latency of different methods.

First, G-Learned Index exhibits an average latency of less than 0.02*ms*. Although G-Learned Index targets high-throughput scenarios, its latency is close to that of ALEX and PGM among most workloads. For *dict* dataset, G-Learned Index can provide even shorter latency. This suggests that the batch query strategy does not significantly affect the latency, even at batch granularity.

Second, we can see that the GPU parallelism can provide much lower latency than the CPU parallel version. One reason is that the GPU is equipped with GDDR6 memory, which has much higher bandwidth and lower latency. Another reason is that workloads are scheduled more evenly on different GPU SMs.

Third, G-Learned Index latencies are similar across datasets. In fact, its reassuring latency on datasets of various sizes is mainly due to the predictable and balanced performance of various queries on GPU kernels, in which minimum branch divergence occurs, and thus few additional time is needed to wait for the last thread in a *warp* to finish its job. In contrast, the structure of B-Tree never guarantees a fixed search path from the root down to the leaves, leading to huge branch divergence.

Applying G-Learned Index to other scenarios. G-Learned Index can be applied to many applications. For the case of web search on dataset *Street*, we have an 16.6× faster end-to-end total execution. For the use case of feature store retrieval on one million taxi rides in New York City [1], suppose we need to select 70% of the

[1] A proven loose bound is $\log(N \times 1/(2\epsilon))$, but empirically quantified investigation shows much better results. Our case here is a compelling justification.

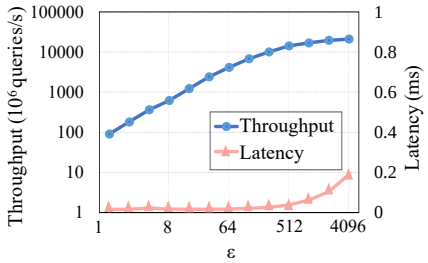

Figure 6: Performance of G-Learned Index with various batch sizes.

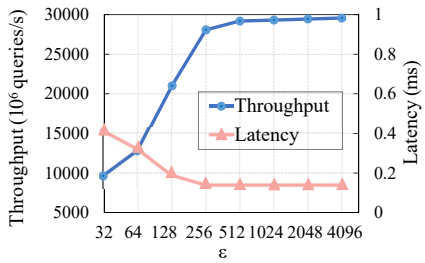

Figure 7: Influence of ϵ on throughput and latency.

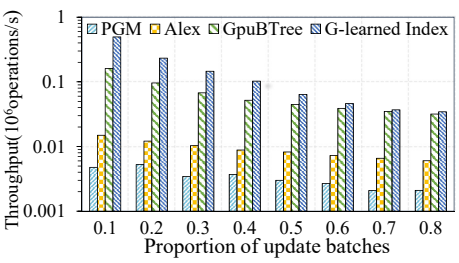

Figure 8: Performance of random updates of different methods.

records as training set randomly; with concurrent batch indexing, the end-to-end processing time speedup is 11.3×.

6.3 Design Tradeoffs

Batch size analysis. Next, our navigation is concerned with the effect of different batch sizes, namely the number of queries completed within a batch. Because a large number of queries arrive at a close time, we organize these queries at batch granularity. Combining queries into a batch aids in making full usage of computing cores on GPU. To investigate the effect of batch size on performance, we vary the batch size from 1KB to 4096KB, and measure the throughput and latency of G-Learned Index.

We show the performance based on various batch sizes in Figure 6. We have the following observations. First, the throughput increases as the batch size grows. When the batch size is less than 512KB, we can see that the throughput grows steadily. Following that, the growth of throughput slows down, indicating that the utilization of parallel GPU resources is close to saturation, and there is contention for memory access that arises at high batch sizes. Second, the latency remains low until the batch size reaches 512KB. After that, the newly arrived batch cannot be processed directly, and the processing time increases. Accordingly, there is a noticeable difference in latency after 512KB. Third, we conclude that properly increasing the batch size when resources are abundant can increase the throughput without affecting the latency.

Influence of different configurations. We report the influence from different ϵ configurations on ten million keys in Figure 7. First, we can see that the throughput grows dramatically when ϵ increases from 32 to 256. This is because a larger ϵ leads to a significant decrease in the number of *segments* at the bottom level, which saves a considerable amount of time during data retrieval and query execution on *conciseQuery*. Although it takes more time for binary search on GPU to find the real position of a key, this time increase accounts for only a small portion of the total time consumed. Therefore, the time saved is not hidden. When ϵ is larger than 256, the throughput does not increase significantly. The reason is that further increases in ϵ have less of an impact even as the number of segments decreases. Additionally, latencies behave in the opposite direction to throughputs. The latency starts out high for $\epsilon = 32$, and gradually decreases until it reaches 256. This is due to faster query execution when ϵ becomes larger.

Influence of memory hierarchy. Each GPU features a memory hierarchy. We investigate the memory hierarchy by explicitly assigning structures of segment levels for the *1e6* dataset with a varying ϵ to specific memory types. We measure the query time and show the performance results in Table 1. Experiments show that storing the index structures in shared memory outperforms global memory by 5% to 18%, and up to 7% over constant memory. The reason is that shared memory, also known as programmer-controlled cache on the GPU, can provide faster read and write bandwidths. Besides, the pre-fetch from global memory to shared memory guarantees a maximum degree of coalescing.

Table 1: Query time with regard to different types of memories on the GPU (*us*).

Memory Type	ϵ			
	32	64	128	256
global memory	17.184	17.152	16.64	14.88
constant memory	15.104	15.104	14.4	14.176
shared memory	15.808	14.016	13.888	14.112

6.4 Random Updates

To measure the performance of random updates in G-Learned Index, we generate a dataset of *1e7* 32 bit integers from $u(0, 2^{32})$ and simulate a dynamic scenario where 10^7 operations are randomly generated. Operations of insertions, deletions, and queries are executed in batches of 2^{12}. Among these batches, we vary the proportion of update operations to explore the performance of G-Learned Index in different situations. We set ϵ to 128, and compare G-Learned Index to the dynamic PGM-index, ALEX, and GPU B-Tree. We show the comparison results in Figure 8. The horizontal axis represents the different proportions of update operations. Figure 8 shows that G-Learned Index outperforms PGM, ALEX, and GPU B-Tree by an average of 32×, 11×, and 1.6× in terms of throughput, respectively.

7 CONCLUSION

This paper proposes the first GPU-based learned index to enable efficient learned indexes on GPUs. It shows how learned indexes have been materialized on GPUs, and discusses the primary challenges in applying GPU in learned indexes. This is addressed by efficient performance-guided bandwidth utilization, optimizations of multi-thread organization and memory dependency removal following the heterogeneous model accompanied with pre-trained parametric selection and on-the-fly strategy adaptation. Moreover, we provide a library to help users adopt our work in GPU-based data management systems. G-Learned Index achieves an average of 3050× speedup over the state-of-the-art method (and 1880× of its parallel version). These experiments show that using GPU in conjunction with machine learning in the index of database systems has tremendous promise.

REFERENCES

[1] 2021. A Billion Taxi Rides in Redshift. https://tech.marksblogg.com/billion-nyc-taxi-rides-redshift.html
[2] 2022. Black Friday Figures. https://black-friday.global/
[3] 2022. Feature Store for ML. https://www.featurestore.org/
[4] 2022. How Much Data Is Created Every Day? https://seedscientific.com/how-much-data-is-created-every-day/
[5] Karolina Alexiou, Donald Kossmann, and Per-Åke Larson. 2013. Adaptive Range Filters for Cold Data: Avoiding Trips to Siberia. *Proc. VLDB Endow.* (2013).
[6] Saman Ashkiani, Shengren Li, Martin Farach-Colton, Nina Amenta, and John D Owens. 2018. GPU LSM: A dynamic dictionary data structure for the GPU. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 430–440.
[7] Muhammad A. Awad, Saman Ashkiani, Rob Johnson, Martín Farach-Colton, and John D. Owens. 2019. Engineering a High-Performance GPU B-Tree. In *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming* (Washington, District of Columbia) *(PPoPP '19)*. Association for Computing Machinery, New York, NY, USA, 145–157. https://doi.org/10.1145/3293883.3295706
[8] Jacquelyn Bulao. 2022. How Much Data Is Created Every Day in 2022? https://techjury.net/blog/how-much-data-is-created-every-day/#gref
[9] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2001. *Introduction to Algorithms* (2nd ed.). The MIT Press.
[10] Intel Corporation. 2022. *Intel® 64 and IA-32 Architectures Software Developer Manuals*.
[11] NVIDIA Corporation. 2018. *CUDA C Programming Guide*.
[12] Jialin Ding, Umar Farooq Minhas, Jia Yu, Chi Wang, Jaeyoung Do, Yinan Li, Hantian Zhang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, David Lomet, and Tim Kraska. 2020. ALEX: An Updatable Adaptive Learned Index. In *Proceedings of the 2020 ACM SIGMOD* (Portland, OR, USA). Association for Computing Machinery, 969–984.
[13] Jialin Ding, Vikram Nathan, Mohammad Alizadeh, and Tim Kraska. 2020. Tsunami: A Learned Multi-dimensional Index for Correlated Data and Skewed Workloads. *Proc. VLDB Endow.* (2020).
[14] Lu Fan, Qimai Li, Bo Liu, Xiao-Ming Wu, Xiaotong Zhang, Fuyu Lv, Guli Lin, Sen Li, Taiwei Jin, and Keping Yang. 2022. Modeling User Behavior with Graph Convolution for Personalized Product Search. In *Proceedings of the ACM Web Conference 2022* (Virtual Event, Lyon, France) *(WWW '22)*.
[15] Qi Fan and Qingyang Wang. 2015. Performance Comparison of Web Servers with Different Architectures: A Case Study Using High Concurrency Workload. In *2015 Third IEEE Workshop on Hot Topics in Web Systems and Technologies (HotWeb)*. 37–42. https://doi.org/10.1109/HotWeb.2015.11
[16] Paolo Ferragina and Giorgio Vinciguerra. 2020. The PGM-Index: A Fully-Dynamic Compressed Learned Index with Provable Worst-Case Bounds. *Proc. VLDB Endow.* (2020).
[17] E. Furst, Mark Henry Oskin, and B. Howe. 2017. Profiling a GPU database implementation: a holistic view of GPU resource utilization on TPC-H queries. (2017).
[18] Alex Galakatos, Michael Markovitch, Carsten Binnig, Rodrigo Fonseca, and Tim Kraska. 2019. FITing-Tree: A Data-Aware Index Structure. In *Proceedings of the 2019 SIGMOD* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1189–1206. https://doi.org/10.1145/3299869.3319860
[19] Goetz Graefe. 2006. B-Tree Indexes, Interpolation Search, and Skew. In *Proceedings of the 2nd International Workshop on Data Management on New Hardware* (Chicago, Illinois) *(DaMoN '06)*. Association for Computing Machinery, New York, NY, USA, 5–es. https://doi.org/10.1145/1140402.1140409
[20] Randall A Hughes, Aleksandr E Miklos, and Andrew D Ellington. 2011. Gene synthesis: methods and applications. In *Methods in enzymology*. Vol. 498. Elsevier, 277–309.
[21] Krzysztof Kaczmarski. 2011. Experimental B+-tree for GPU. *ADBIS (2)* 11 (2011).
[22] Krzysztof Kaczmarski. 2012. B + -Tree Optimized for GPGPU. In *On the Move to Meaningful Internet Systems: OTM 2012*, Robert Meersman, Hervé Panetto, Tharam Dillon, Stefanie Rinderle-Ma, Peter Dadam, Xiaofang Zhou, Siani Pearson, Alois Ferscha, Sonia Bergamaschi, and Isabel F. Cruz (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 843–854.
[23] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. 2019. SOSD: A Benchmark for Learned Indexes. arXiv:1911.13014 [cs.DB]
[24] Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. 2020. RadixSpline: A Single-Pass Learned Index. In *Proceedings of aiDM* (Portland, Oregon) *(aiDM '20)*. Association for Computing Machinery, Article 5, 5 pages.
[25] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data* (Houston, TX, USA) *(SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 489–504. https://doi.org/10.1145/3183713.3196909
[26] Ani Kristo, Kapil Vaidya, Ugur Çetintemel, Sanchit Misra, and Tim Kraska. 2020. The Case for a Learned Sorting Algorithm. In *SIGMOD*.
[27] Jyoti B Kulkarni, AA Sawant, and Vandana S Inamdar. 2011. Database processing by Linear Regression on GPU using CUDA. In *2011 International Conference on Signal Processing, Communication, Computing and Networking Technologies*.
[28] Jurek Leonhardt, Koustav Rudra, Megha Khosla, Abhijit Anand, and Avishek Anand. 2022. Efficient Neural Ranking Using Forward Indexes. In *Proceedings of the ACM Web Conference 2022 (WWW '22)*. Association for Computing Machinery, 266–276.
[29] Pengfei Li, Yu Hua, Jingnan Jia, and Pengfei Zuo. 2021. FINEdex: A Fine-Grained Learned Index Scheme for Scalable and Concurrent Memory Systems. *Proc. VLDB Endow.* 15, 2 (oct 2021), 321–334. https://doi.org/10.14778/3489496.3489512
[30] Xin Li, Jingdong Li, and Xiaoling Wang. 2019. ASLM: Adaptive Single Layer Model for Learned Index. In *Database Systems for Advanced Applications*, Guoliang Li, Jun Yang, Joao Gama, Juggapong Natwichai, and Yongxin Tong (Eds.). Springer International Publishing, Cham, 80–95.
[31] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym. 2008. NVIDIA Tesla: A Unified Graphics and Computing Architecture. *IEEE Micro* 28, 2 (2008), 39–55. https://doi.org/10.1109/MM.2008.31
[32] Witold Litwin. 1980. Linear Hashing: A New Tool for File and Table Addressing. In *Proceedings of the Sixth International Conference on Very Large Data Bases - Volume 6*.
[33] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2021. Epoch-based commit and replication in distributed OLTP databases. *Proceedings of the VLDB Endowment* (2021).
[34] Ziyan Luo, Linfeng Zhao, Wei Cheng, Sihao Chen, Qi Chen, Hui Xue, Haidong Wang, Chuanjie Liu, Mao Yang, and Lintao Zhang. 2021. Match Plan Generation in Web Search with Parameterized Action Reinforcement Learning. In *Proceedings of the Web Conference 2021 (WWW '21)*. New York, NY, USA, 1040–1052.
[35] Ziyan Luo, Linfeng Zhao, Wei Cheng, Sihao Chen, Qi Chen, Hui Xue, Haidong Wang, Chuanjie Liu, Mao Yang, and Lintao Zhang. 2021. Match Plan Generation in Web Search with Parameterized Action Reinforcement Learning. In *Proceedings of the Web Conference 2021 (WWW '21)*. 1040–1052.
[36] Darko Makreshanski, Jana Giceva, Claude Barthels, and Gustavo Alonso. 2017. BatchDB: Efficient isolated execution of hybrid OLTP+ OLAP workloads for interactive applications. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 37–50.
[37] Maleeha Najam, Raihan Rasool, Hafiz Ahmad, Usman Ashraf, and Asad Malik. 2019. Pattern Matching for DNA Sequencing Data Using Multiple Bloom Filters. *BioMed Research International* 2019 (04 2019), 1–9. https://doi.org/10.1155/2019/7074387
[38] Joseph O'Rourke. 1981. An On-Line Algorithm for Fitting Straight Lines between Data Ranges. *Commun. ACM* 24, 9 (Sept. 1981), 574–578. https://doi.org/10.1145/358746.358758
[39] Laurel Orr, Atindriyo Sanyal, Xiao Ling, Karan Goel, and Megan Leszczynski. 2021. Managing ML pipelines: feature stores and the coming wave of embedding ecosystems. *Proceedings of the VLDB Endowment* 14, 12 (2021), 3178–3181.
[40] Varun Pandey, Andreas Kipf, Thomas Neumann, and Alfons Kemper. 2018. How Good Are Modern Spatial Analytics Systems? *Proc. VLDB Endow.* (2018).
[41] Wenwen Qu, Xiaoling Wang, Jingdong Li, and Xin Li. 2019. Hybrid Indexes by Exploring Traditional B-Tree and Linear Regression. In *Web Information Systems and Applications*, Weiwei Ni, Xin Wang, Wei Song, and Yukun Li (Eds.). Springer International Publishing, Cham, 601–613.
[42] Aunn Raza, Periklis Chrysogelos, Angelos Christos Anadiotis, and Anastasia Ailamaki. 2020. Adaptive HTAP through elastic resource scheduling. In *SIGMOD*.
[43] A. Redd, K. Khin, and A. Marini. 2019. Fast ES-RNN: A GPU Implementation of the ES-RNN Algorithm. (2019).
[44] Christopher Root and Todd Mostak. 2016. MapD: A GPU-Powered Big Data Analytics and Visualization Platform. In *ACM SIGGRAPH 2016 Talks*.
[45] B. G. Soos, A. Rak, J. Veres, and G. Cserey. 2008. GPU powered CNN simulator (SIMCNN) with graphical flow based programmability. In *International Workshop on Cellular Neural Networks & Their Applications*.
[46] Peter Van Sandt, Yannis Chronis, and Jignesh M. Patel. 2019. Efficiently Searching In-Memory Sorted Arrays: Revenge of the Interpolation Search?. In *SIGMOD*.
[47] H. Xu, L. Zeng, X. Cai, and S. Li. 2009. GPU-accelerated feature extraction and multi-resolution visualization for complex 3D fluid field. *Jisuanji Fuzhu Sheji Yu Tuxingxue Xuebao/Journal of Computer-Aided Design and Computer Graphics* 21, 7 (2009), 893–899.
[48] Simin You, Jianting Zhang, and Le Gruenwald. 2013. Parallel Spatial Query Processing on GPUs Using R-Trees. In *Proceedings of the 2nd ACM SIGSPATIAL*.
[49] Puxuan Yu, Hongliang Fei, and Ping Li. 2021. Cross-Lingual Language Model Pretraining for Retrieval. In *Proceedings of the Web Conference 2021 (WWW '21)*. Association for Computing Machinery, New York, NY, USA.
[50] Kai Zeng, Sameer Agarwal, and Ion Stoica. 2016. IOLAP: Managing uncertainty for efficient incremental OLAP. In *Proceedings of the 2016 international conference on management of data*. 1347–1361.

A PROOFS

To theoretically support parallel index construction, we describe our observation by first giving the following definition.

Definition 3. Let P be the point set $\{(k_i, rank(k_i))\}$ sorted according to their x-coordinates. A segment generation problem (SGP) takes the sorted points in P and an initial status (st, le) as inputs. The le clarifies the covering range of the current segment. Namely, it starts at the le^{th} point, and st denotes the next point to process. Accordingly, the SGP (A, st, le) outputs the minimum number of segments needed in PLA-model for covering the points starting from the st^{th} with a current segment status le.

Obviously, constructing the PLA-model on A can be viewed as solving the SGP$(P, 1, 1)$, where $P = \{(k_i, rank(k_i))\}$ for all k_i in A, and a segment starts from and covers only the first point.

Lemma 1. *Satisfying the optimization principle and Markov property, we design an $O(n^3)$ dynamic programming (DP) strategy by defining $dp_{i,j}$ as follows. Given a SGP$(P, 1, 1)$, a minimum of $dp_{i,j}$ segments is needed to cover the first i points, with the current segment starting from the point whose $rank = j$. All $dp_{*,*}$ are initialized to ∞ except $dp_{st,le} = 1$. We also maintain a $f_i = \min\{dp_{i,*}\}$ for each i. The algorithm runs by following $dp_{i,j} = dp_{i-1,j}$ if the segment starting from the point j can be extended to i. Otherwise, $dp_{i,j} = \min\{f_k + 1\}$ with $k < i$. In this case, the $(k+1)^{th}$ point to the ith point must satisfy the condition to form a segment, because we are forming a new segment. Note that f_n represents the final result.*

Both the greedy algorithm and the above DP strategy [18, 38] achieve the same optimal number of segments. We consider the problem from the DP strategy.

We focus on the matrix C_1 where $c_{i,j} = dp_{i,j}$ for SGP1(P, st, le). We consider the corresponding matrix for SGP2(P, st, le_2) with $le_2 > le$. First, we obtain a copy of C_1 for C_2. Then, we set dp_{st,le_2} to 1 and update C_2. Now, the dp matrix C_2 for SGP2 is going to, in a sense, obtain results no worse than C_1. The copy operation is reasonable because the same state transition routes hold for two matrices. At this time, a looser initial status of $dp_{st,le_2} = 1$ is possibly able to bring a better result in f_n. This finding assists to bring theorem 1.

A.1 Proof of Theorem 1

Proof. Consider situations where $\omega = 1$ and it is easy to apply the same procedure to prove for $\omega > 1$. If the split happens where it should be, no extra segments are encountered. In the other case, when performing PLA-model, suppose we are now at the $(cur-1)^{th}$ point with a current segment starting from the lo^{th} point. However, we cannot extent the segment to the cur^{th} point because a split happens between the $(cur-1)^{th}$ and cur^{th} points. The post-split situation fits into a SGP1, where $st = cur$ and $le = cur$. Instead, we consider the original construction process, if there is no split, as a SGP2(cur, lo). We obtain an additional $\omega = 1$ segment by deliberately disconnecting the current segment in the parallel PLA-model, regarding it as an extra one compared to the original PLA-model. By this way, we are turning a SGP2 to SGP1. Since SGP1 achieves results no worse than SGP2, we prove the theorem that at most an extra of $\omega = 1$ segment is created by the parallel PLA-model. □

A.2 Proof of Theorem 2

Proof. A factor of c, which is more than $2\epsilon_2$, represents the reduction in the number of segments of the lower level of segments over the upper. Consequently, a total number $L = O(\log_c m)$ of levels are determined within $\sum_{l=0}^{L} m/c^l = \Theta(m)$ space required in GPUs. For a query task, amortized execution time is bounded by L binary searches with $L-1$ on GPU and 1 search on CPU. Specifically, $L-1$ binary searches are conducted on GPU over the $L-1$ intervals with the size of at most $2\epsilon_2 + 1$. One search is performed on CPU over the range whose size is at most $2\epsilon_1 + 1$. As for the I/O costs, besides one binary search on CPU required for each query, I/O complexity also involves the extra expenses for loading segments from CPU and the transmission of results of batch queries back from GPU. In case of memory transactions on GPU, each thread is responsible for retrieving data that binary searches need in the device memory. □

B STRATEGY FOR FASTER QUERY EXECUTION

As shown in Figure 9, we find that for specific cases, such as segments with only three levels, removing the upper levels and conducting direct traversal check search on the bottom level can produce better query time results. This is because we need fewer memory transactions, and more importantly, perform the exact same numerical comparison between threads. For this reason, we build the index and use an auto-tuning *lookahead strategy selection* process. The process of selection is as follows. First, we select a sample query at random for the test query. Second, we track the time spent on *completeQuery*, namely indexing this sample query on all levels of segments. Third, we time the *conciseQuery*, i.e., conducting direct comparisons on each key in the last level to find the right-most one smaller than or equivalent to k. Finally, we compare the time spent in the second and third steps and select a more efficient strategy.

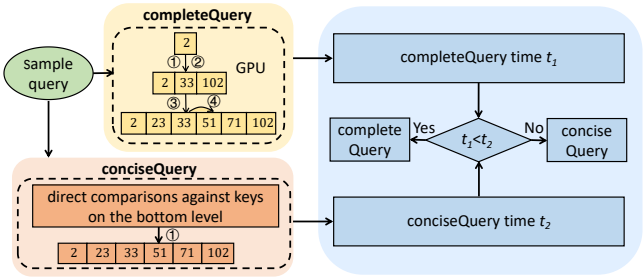

Figure 9: Lookahead strategy selection module. A query sample is selected and follows *completeQuery* and *conciseQuery*.

Example. In Figure 2, we set ϵ_1 and ϵ_2 to 1. The upper three levels of segments are generated and placed on the GPU, and the array A stays on CPU. Before processing batch queries, we initially pick up the first batch as a sample query set. Assume the batch size is 2^{12}. Then, we launch 2^{12} threads for the sample query set where each thread solves one query. We next launch *completeQuery* and *conciseQuery* in turn as shown in Figure 9. Suppose there is a thread assigned to index $x = 58$. For *completeQuery*, the thread follows the four procedures as discussed in the example of the PGM-index in Section 2.1, and determines the accurate position in the last level of segments denoted by 51. For *conciseQuery*, the only procedure

is that the thread directly conducts comparison against keys on the last level of segments and finds that segment. Then, using the segment denoted by 51, the thread returns 12 to CPU as the estimated position of x in A. The processing times of t_1 and t_2 are recorded for *completeQuery* and *conciseQuery*, respectively. In this case t_1 is larger than t_2, so G-Learned Index chooses *conciseQuery* as the query strategy. G-Learned Index then executes batch queries using *conciseQuery*. GPU processes queries in a batch and returns the estimated positions to CPU. Collecting all the 2^{12} estimated positions, the CPU processor then conducts binary searches. For the query $x = 58$ for instance, CPU searches among $A[12-\epsilon, 12+\epsilon]$ to determine the final position of x in A.

C SPACE SAVINGS OF G-LEARNED INDEX

In this part, we investigate the space savings of G-Learned Index by tracking the footprint of our implementation, which to a large extent depends on the amount of space required to store the bottom level of segments. We compare the size of G-Learned Index with that of GPU-based B-Tree.

Overall, experiments show that given $\epsilon = 128$, for the four real-world datasets, G-Learned Index saves space by 4258×, 528×, 783× and 11333× respectively. In addition, for the synthetic datasets, G-Learned Index saves 4 orders of magnitudes space on average. The reason is that *segments* of the index are compact structures that take up space sub-linear in the number of keys in a dataset.

To explore the influence of ϵ on the space savings, we vary ϵ from 32 to 128, 512, and 2048 over the eight datasets. The average space savings for different ϵ are 1130×, 1.4e4×, 1.5e5× and 9.4e4×, respectively. The space benefits increase along with ϵ. The reason is that the PLA model generates *segments* dependent on choices of ϵ, which denotes the tolerance of precision that is allowed when using a *segment* to approximate the position of a key. Specifically, as ϵ increases, fewer *segments* are needed resulting in greater space savings.

D INDEX CONSTRUCTION

D.1 Analysis of Parameters for Index Construction

G-Learned Index is separated and placed on CPU and GPU. We give analysis for choosing appropriate ϵ for the respective index components on CPU and GPU in the following two steps. In the first step, we look into the procedures it takes for indexing a key k in the model and how the choice of ϵ can affect those procedures. In the second step, we provide detailed analysis to support the division of the procedures theoretically.

In the first step, we divide the procedures of a searching process for k into two types. One ($type_1$ procedures in Figure 3) consists of two procedures: 1) a mapping from the corresponding segment in the last level to a predicted position in A with an error of ϵ on the GPU, and 2) a CPU procedure of pinpointing the final position of k. The other ($type_2$ procedures in Figure 3) contains procedures beginning at the root segment (which completes a level) and ending with one of the segments at the last level. These procedures are all conducted on the GPU. To sum up, a G-Learned Index structure is made up of these two types of procedures, as well as the two parts discussed in heterogeneous structure preparation.

In the second step, we investigate the effect of varying ϵ on $type_1$ and $type_2$ procedures. In the case of $type_1$ procedures, we first consider the factors of space occupancy. A large ϵ can reduce the number of segments at the bottom level, saving load costs. Furthermore, fewer segments can be stored in the limited GPU shared memory, resulting in more efficient data accesses and thus better time performance. In the case of $type_2$ procedures, we focus on the factors of time efficacy. A small ϵ is required because a small ϵ indicates a limited positioning range, resulting in fewer operation discrepancies and less waiting time. This can eventually lead to the optimal branch divergence situations, which can boost GPU efficiency. Based on the previous discussions, we find that the two procedures have opposing preferences for ϵ. Therefore, we design an adaptive auto-fitting module for two separate ϵs corresponding to the two types of procedures. We define ϵ_1 as the ϵ used for generating the last level of segments from array A, and define ϵ_2 for internally building segments recursively from the bottom level up to the root segment. Theoretically, we prefer a large ϵ_1 and a small ϵ_2 (in fact, $\epsilon_2 = 4$ yields a competent result). Further tuning and investigation of ϵ_1 are explored in Section 6.

D.2 Performance of Index Construction

As discussed in Section 5.1, G-Learned Index can be built on GPU in parallel. The index construction process on GPU of G-Learned Index is much faster than that on CPU. On average, G-Learned Index is 5.54× and 3.05× faster than PGM (parallel) and ALEX, respectively. Moreover, we find that the index construction of G-Learned Index on large datasets exhibits higher benefits. For example, G-Learned Index achieves 8.35× speedup in construction on the "*1e8*" dataset and 2.28× on the "*1e5*" dataset over PGM.